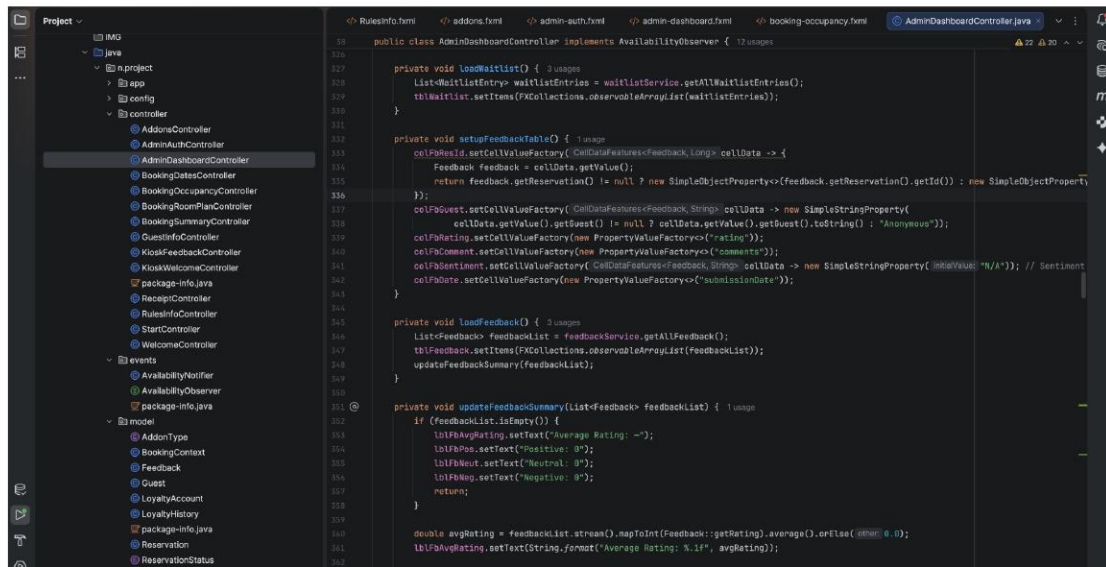


Project

Application Development

By Syed Nabeel Raza Dec 5, 2025

| APD545NCC | Prof. Mahboob Ali



By Syed Nabeel Raza Dec 5, 2025

1. Introduction

1.1 Project Overview

1.2 Objectives

2. System Overview

2.1 Main Modules

2.1.1 Guest Kiosk

2.1.2 Administrative Dashboard

3. Technology Stack

3.1 Core Technologies

3.2 Rationale for Technology Choices

4. System Architecture

4.1 3-Tier Architecture

4.1.1 Presentation Tier (UI & Controllers)

4.1.2 Business Logic Tier (Service Layer)	4
4.1.3 Data Access Tier (Repository Layer)	4
4.2 Cross-Cutting Concerns	5
4.2.1 Dependency Injection	5
4.2.2 Logging	5
4.2.3 Scene Management	5
5. MVC, DI, and ORM	5
5.1 Model-View-Controller (MVC)	5
5.2 Dependency Injection (DI)	6
5.3 Object-Relational Mapping (ORM)	6
6. Data Model and Relationships	6
6.1 Core Entities	6
6.2 Relationship Characteristics	7
7. Design Pattern Usage	8
7.1 Singleton	8
7.2 Factory	8
7.3 Observer	8
7.4 Strategy	9
7.5 Decorator (Conceptual)	9
8. Business Rules	9
8.1 Occupancy and Room Allocation	9
8.2 Booking Validation	9
8.3 Pricing and Discounts	9
8.4 Loyalty Program Rules	10
8.5 Feedback Rules	10
9. Security and Logging	10
9.1 Authentication and Roles	10
9.2 Logging	10
9.3 Exception Handling	11
10. Reporting and Export	11
10.1 Report Types	11
10.2 Export Capabilities	11
11. Challenges, Learnings, and Future Work	12
11.1 Technical Challenges	12
11.2 Personal Learnings	12
11.3 Future Enhancements	12
12. Conclusion	13

1. Introduction

1.1 Project Overview

The Xeveora Hotel Management System is a desktop-based application designed to streamline hotel operations by integrating core management functions into a single platform. It provides:

- A guest-facing kiosk interface for bookings and feedback.
- An administrative dashboard for hotel staff to manage reservations, payments, guests, and reports.

The primary objective is to:

- Improve guest experience through self-service options.
- Increase staff efficiency in daily operations.
- Provide actionable insights through reporting and analytics.

1.2 Objectives

The key objectives of the system are:

- Centralize hotel management operations in a unified solution.
 - Reduce manual workload and potential human errors.
 - Enhance decision-making through structured data and reports.
 - Provide a scalable and maintainable architecture based on industry best practices.
-

2. System Overview

2.1 Main Modules

The system is divided into two major functional modules:

2.1.1 Guest Kiosk

Features include:

- **Guided Booking Wizard**
 - Step-by-step booking flow.

- Selection of check-in/check-out dates.
- Occupancy specification (adults and children).
- Room plan selection based on availability and occupancy rules.
- **Feedback Submission**
 - Simple form for rating (1–5), category, and comments.
 - Optional linkage to a specific reservation.
 - Option for guests to indicate if they can be contacted.
- **Reservation Lookup (Planned)**
 - Guests can view details of existing reservations using a reservation ID.
- **Hotel Services Information (Planned)**
 - Display of hotel amenities and available services.

2.1.2 Administrative Dashboard

Features include:

- **Reservation Management**
 - View, filter, and search all reservations.
 - Create, update, cancel, and check out reservations.
- **Payment Processing**
 - Record payments and track outstanding amounts.
 - Apply discounts using configurable billing strategies.
- **Guest & Loyalty Management**
 - Manage guest profiles and contact information.
 - Points-based loyalty program with earning and redemption logic.
- **Waitlist Management**
 - Add and track guests waiting for room availability.
- **Feedback Management and Analysis**
 - View all feedback entries.
 - Summarize average ratings and comments.
- **Reporting and Export**
 - Revenue reports.
 - Occupancy reports.
 - Feedback summaries.
 - Export to CSV/TXT for external analysis.
- **System Activity Logging**

- Display log entries for monitoring and troubleshooting.
-

3. Technology Stack

3.1 Core Technologies

- **Programming Language:** Java 17
- **UI Framework:** JavaFX with FXML
- **Dependency Injection:** Google Guice
- **ORM / Persistence:** Hibernate with JPA annotations
- **Database:** H2 (file-based embedded database)
- **Logging:** Apache Log4j 2

3.2 Rationale for Technology Choices

- JavaFX provides a rich desktop UI framework suitable for kiosk and admin interfaces.
 - Guice promotes loose coupling and testability through dependency injection.
 - Hibernate simplifies database access and schema management using ORM.
 - H2 offers an embedded, lightweight database that is easy to set up and suitable for local deployments.
 - Log4j 2 enables flexible and configurable logging across the entire application.
-

4. System Architecture

4.1 3-Tier Architecture

The system follows a 3-tier architecture to ensure clear separation of concerns and maintainability.

4.1.1 Presentation Tier (UI & Controllers)

- Implemented with JavaFX and FXML.
- Consists of view definitions (.fxml files) and corresponding controller classes.
- Responsibilities:

- Handle user interactions (button clicks, form submissions).
- Perform input validation.
- Communicate with service layer to execute business operations.

Examples:

- Views: kiosk_welcome.fxml, admin-dashboard.fxml, booking-room-plan.fxml, feedback.fxml.
- Controllers: KioskFeedbackController, BookingRoomPlanController, AdminDashboardController.

4.1.2 Business Logic Tier (Service Layer)

- Implemented in the `n.project.service` package.
- Responsibilities:
 - Encapsulate application business rules.
 - Orchestrate workflows (e.g., booking creation, cancellation, loyalty updates).
 - Remain independent of UI and specific database implementation.

Examples:

- BookingService – manages reservations and room availability.
- FeedbackService – handles feedback creation and retrieval.
- LoyaltyService – manages loyalty points and transaction rules.
- PaymentService – applies pricing and discount strategies.

4.1.3 Data Access Tier (Repository Layer)

- Implemented in the `n.project.repository` package.
- Responsibilities:
 - Communicate with the database using Hibernate.
 - Provide CRUD operations for entities.
 - Hide SQL/HQL details from services.

Examples:

- ReservationRepository
- RoomRepository
- GuestRepository

- FeedbackRepository

4.2 Cross-Cutting Concerns

4.2.1 Dependency Injection

- Managed by Google Guice.
- Dependencies are provided via constructor injection (@Inject).
- AppModule configures bindings for controllers, services, repositories, and utilities.
- BookingContext is managed as a shared state object (similar to a session) across the booking wizard.

4.2.2 Logging

- Log4j 2 is used to log:
 - Informational events (e.g., navigation, successful operations).
 - Warnings (e.g., missing data, invalid input).
 - Errors and exceptions (stack traces, failures).
- Hibernate SQL logging is enabled for debugging database interactions.

4.2.3 Scene Management

- SceneManager is a central utility responsible for:
 - Loading FXML scenes.
 - Managing stage transitions.
 - Opening modal windows (e.g., rules and policy dialogs).
 - Controllers rely on SceneManager rather than manually loading FXML files.
-

5. MVC, DI, and ORM

5.1 Model-View-Controller (MVC)

- **Model:**
 - Entity classes under n.project.model (e.g., Guest, Reservation, Room, Feedback).
 - Represent application data and business state.
- **View:**

- FXML files define the visual layout and controls of each screen.
- **Controller:**
 - Java classes under `n.project.controller`.
 - Bind to FXML via `fx:controller`.
 - Handle logic between the View and Model through Services.

5.2 Dependency Injection (DI)

- Controllers and services do not instantiate their dependencies directly.
- Guice injects instances at runtime.
- Benefits:
 - Loose coupling.
 - Easier unit testing (mock services and repositories).
 - Centralized configuration of bindings.

5.3 Object-Relational Mapping (ORM)

- Hibernate maps Java entities to database tables via JPA annotations:
 - `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, etc.
 - Relationship annotations: `@ManyToOne`, `@OneToMany`, `@ManyToMany`, `@OneToOne`.
 - Developers work with Java objects instead of raw SQL queries.
-

6. Data Model and Relationships

6.1 Core Entities

- **Guest**
 - Represents a hotel customer.
 - Key fields: `id`, `firstName`, `lastName`, `email`, `phone`.
 - Relationships:
 - `@OneToMany(mappedBy = "guest")` with `Reservation`.
 - `@OneToOne(mappedBy = "guest", cascade = CascadeType.ALL)` with `LoyaltyAccount`.
- **Reservation**

- Represents a room booking.
- Fields: id, checkInDate, checkOutDate, numAdults, numChildren, status, totalPrice, amountPaid.
- Relationships:
 - @ManyToOne Guest (non-null).
 - @ManyToMany Rooms via join table.
- **Room**
 - Represents a physical hotel room.
 - Fields: id, roomNumber, roomType, basePrice.
- **Feedback**
 - Represents guest feedback.
 - Fields: id, rating, category, comments, submissionDate, canContact.
 - Relationships:
 - @ManyToOne Guest (optional).
 - @OneToOne Reservation (optional).
- **LoyaltyAccount**
 - Tracks guest loyalty points.
 - Fields: id, points, loyaltyNumber.
 - Relationship:
 - @OneToOne Guest.
- **LoyaltyHistory**
 - Records point earning and redemption.
 - Relationship:
 - @ManyToOne LoyaltyAccount.
- **WaitlistEntry**
 - Represents a waitlisted request for a room when capacity is full.

6.2 Relationship Characteristics

- **Cascading:**
 - Guest → LoyaltyAccount uses CascadeType.ALL, so saving a Guest also saves the associated loyalty account.
 - Other cascades are managed manually in the service layer for finer control.
- **Fetching:**

- Collections (@OneToMany, @ManyToMany) are generally LAZY to avoid unnecessary loading.
 - @ManyToOne and @OneToOne defaults are EAGER, which is acceptable for primary references.
 - **Validation:**
 - Database constraints such as @Column(nullable = false) enforce required fields.
 - Controller-level validation ensures user inputs are checked before reaching the service layer.
-

7. Design Pattern Usage

7.1 Singleton

- BookingContext acts as a shared state object across the booking wizard.
- Managed as a singleton via DI, so all related controllers share the same instance.

7.2 Factory

- RoomFactory encapsulates creation logic for Room objects based on RoomType and pricing details.
- This avoids duplicating construction logic in multiple services.

7.3 Observer

- Implemented to keep views in sync with data changes:
 - **Subject:**
 - AvailabilityNotifier broadcasts changes when reservations are created, cancelled, or checked out.
 - **Observers:**
 - AdminDashboardController and other listeners implement AvailabilityObserver.
 - When notified, they refresh reservations and availability views.

7.4 Strategy

- PaymentService uses different billing strategies:
 - StandardBillingStrategy for regular pricing.
 - DiscountBillingStrategy for discounted calculations.
- The strategy can be switched at runtime without changing the PaymentService itself.

7.5 Decorator (Conceptual)

- DiscountBillingStrategy behaves similarly to a Decorator by adding discount logic on top of base pricing behavior.
-

8. Business Rules

8.1 Occupancy and Room Allocation

- Business logic in BookingRoomPlanController:
 - Suggests a room plan based on number of adults and children.
 - Considers room type capacities defined in RoomType.
 - Ensures suggestions respect both guest count and room availability.

8.2 Booking Validation

- BookingDatesController enforces:
 - Check-out date must be strictly after check-in date.
 - Dates cannot be in the past.
- Validation occurs as the user interacts with date pickers.

8.3 Pricing and Discounts

- Total price is calculated based on:
 - Room type and base price.
 - Number of nights.
 - Number and type of rooms.
- Discounts are applied using the configured billing strategy.

8.4 Loyalty Program Rules

- Points are earned based on payment amounts (e.g., fixed points per currency unit).
- Redemption rules prevent using more points than available.
- Loyalty history is recorded for auditing and analysis.

8.5 Feedback Rules

- Rating (1–5) is mandatory for feedback.
 - Comments and reservation linkage are optional.
 - Guests can indicate whether they are open to follow-up contact.
-

9. Security and Logging

9.1 Authentication and Roles

- The main Welcome screen routes users to:
 - Public kiosk flow (no authentication, limited functionality).
 - Admin login screen for privileged access.
- Role concepts:
 - **Guest:** Limited access via kiosk (booking, feedback).
 - **Admin:** Full access to the dashboard (reservations, reports, payments, logs).
- Future enhancements may include:
 - Password hashing and secure credential storage.
 - Role-based access control (RBAC) for more granular permissions.

9.2 Logging

- Log4j 2 provides structured application logging.
- Use cases:
 - Debugging application flow and errors.
 - Monitoring system activities.
 - Auditing sensitive operations.
- SQL logging via Hibernate's `show_sql` helps verify ORM-generated queries.
- Logs are stored in files under a dedicated `/logs/` directory (e.g., `system_logs.log`).

9.3 Exception Handling

- **Controller Layer:**
 - Catches common user input issues (e.g., invalid reservation IDs).
 - Displays friendly messages using labels (e.g., `lblFeedbackStatus`, `lblDateError`).
 - **Service/Repository Layer:**
 - Catches unexpected runtime exceptions.
 - Logs full stack traces with `logger.error()`.
 - **SceneManager:**
 - Wraps FXML loading with try-catch and shows alerts when a view fails to load.
-

10. Reporting and Export

10.1 Report Types

- **Revenue Report:**
 - Summaries of subtotal, tax, and total revenue over a date range.
 - Option to group by period (e.g., day, week, month) depending on configuration.
- **Occupancy Report:**
 - Comparison of total rooms vs. occupied rooms.
 - Calculation of occupancy rates.
- **Feedback Summary:**
 - Listing of feedback entries, ratings, categories, and comments.
 - Aggregated statistics such as average rating.

10.2 Export Capabilities

- CSV export using a generic `CsvExporter` utility for:
 - Reservations table.
 - Reports.
 - Feedback and other tabular data.
 - Log export to TXT for further analysis.
 - Enables external processing in tools like Excel or Google Sheets.
-

11. Challenges, Learnings, and Future Work

11.1 Technical Challenges

- **State Management Across Multiple Scenes:**
 - Sharing state between multiple booking steps initially created tight coupling.
 - Solution: Introduced BookingContext as a shared state object managed via DI.
- **Complexity of Admin Dashboard Controller:**
 - Single controller handling many tabs and operations led to a large class.
 - Current mitigation: Logical grouping and folding of sections.
 - Planned improvement: Split into multiple controllers, one per tab, using `<fx:include>`.
- **Real-Time UI Refresh:**
 - Required for accurate display of reservations and availability after changes.
 - Solution: Implemented Observer pattern with AvailabilityNotifier and observer controllers.

11.2 Personal Learnings

Key learnings from the project include:

- The value of a clean architecture (3-tier, DI, clear separation) in managing complexity.
- Practical experience with Hibernate and JPA, including relationship mapping and fetch strategies.
- Real-world application of design patterns (Observer, Strategy, Factory, Singleton) and their impact on code flexibility and maintainability.

11.3 Future Enhancements

Potential future improvements:

- **Refactoring:**
 - Decompose AdminDashboardController into smaller, focused controllers.
- **Security Hardening:**
 - Implement robust admin authentication with hashing and secure storage.
 - Introduce role-based access control with multiple staff roles.
- **Validation:**

- Add more comprehensive service-level validation to protect data integrity.
 - **Asynchronous Operations:**
 - Use background threads for long-running tasks (e.g., large report generation) to keep the UI responsive.
 - **Testing:**
 - Introduce unit tests (JUnit) and integration tests (with in-memory H2) for services and repositories.
-

12. Conclusion

The Xeveora Hotel Management System demonstrates a full-featured hotel management solution built on a robust Java-based stack. By combining a structured architecture, clear separation of concerns, and well-chosen design patterns, the system:

- Simplifies hotel operations for staff.
- Provides a user-friendly experience for guests.
- Lays a solid foundation for future enhancements, scalability, and maintainability.

This project also provided significant practical exposure to enterprise concepts such as layered architecture, dependency injection, ORM, and pattern-driven design.